Questions de cours

---

1. Dans un environnement réparti, quel est le rôle de la souche côté client ?

La **souche client** (ou *client stub*) agit comme un **représentant local de l'objet distant** sur le site client. Elle masque la complexité de la communication réseau et **permet au client d'appeler des méthodes distantes comme s'il s'agissait de méthodes locales**. Elle s'occupe de la sérialisation des paramètres, de l'envoi de la requête au serveur et de la réception de la réponse.

2. Donner le pseudo-code de la souche côté client.

Exemple de pseudo-code :

```
fonction appeler_méthode_distant(méthode, paramètres):
    message = marshalling(méthode, paramètres)
    envoyer(message, adresse_serveur)
    réponse = recevoir()
    résultat = unmarshalling(réponse)
    retourner résultat
```

Elle réalise donc trois étapes principales :

1. **Marshall** des arguments.
2. **Envoi** de la requête au serveur.
3. **Réception et unmarshall** du résultat.

3. Comment sont les souches client et serveur définies ?

Les souches **client et serveur** sont générées automatiquement à partir de l'interface distante (*remote interface*).

- Dans **RMI (Java)**, elles sont créées à partir de cette interface à l'aide de la commande `rmic` (dans les anciennes versions) ou automatiquement par le compilateur RMI.

- Elles implémentent la **même interface distante**, assurant ainsi la compatibilité entre client et serveur.

4. Dans RMI, que se passe-t-il quand un serveur exporte un objet distant ? Avec quelle commande ? Quels sont les arguments ?

Lorsqu'un **serveur exporte un objet distant**, il rend cet objet **accessible à distance** pour les clients. Cela crée une **souche serveur** qui écoute les appels entrants.

Commande utilisée :

UnicastRemoteObject.exportObject(objet, port);

Arguments :

- `objet` : l'objet distant à exporter (implémentant l'interface distante).
- `port` : le numéro de port sur lequel l'objet sera accessible (ou `0` pour un port choisi automatiquement).

---

5. Dans RMI, qu'est-ce que la commande `lookup` de la classe `Naming` permet de récupérer ?

La méthode : Naming.lookup("rmi://hôte:port/nomObjet");

permet de **récupérer une référence (stub)** vers un **objet distant enregistré** dans le registre RMI (*RMI Registry*). Le client peut ensuite invoquer des méthodes sur cet objet comme s'il s'agissait d'un objet local.

6. Quelles sont les limites des sockets pour développer des applications réparties ?

- Beaucoup de **détails techniques** à gérer (adresses IP, ports, protocoles, synchronisation…).
- **Aucune abstraction** des objets ou méthodes distantes.
- **Pas de mécanismes intégrés** pour le marshalling ou la sérialisation.

- Nécessité d’une **connaissance approfondie** du réseau et des protocoles bas-niveau.
  etc …

7. Donnez la définition de "marshalling" dans le contexte d’applications réparties.

Le **marshalling** est le processus de **conversion et d’emballage** des paramètres d’un appel de méthode (ou d’un objet complet) dans un **format transmissible sur le réseau**.
Il permet de **sérialiser** les données avant leur envoi et de les **désérialiser** à la réception (opération inverse : *unmarshalling*).

8. Quel est le rôle d’une représentation intermédiaire de données ? Que définit-elle ?

Une **représentation intermédiaire de données** (ou *intermediate data representation*) sert à :

- **Assurer l’indépendance des systèmes hétérogènes**, en offrant un format commun pour échanger des données entre machines ayant des architectures ou systèmes différents.

- Elle définit :

    - La **façon dont les données sont codées**
    - Les **règles de conversion** entre la représentation locale et la représentation commune.